Software Requirements Specification
Restaurant App MVP - Full Stack (React Native + Node.js + MongoDB)
Version 2.0 - Assignment 2
Course: MAD Fall 2026 - Prepared by Manahil Fatima - Reg 9712

1. INTRODUCTION

1.1 Purpose
V1 (A1) was frontend-only with mock data in React state and AsyncStorage.
V2 (A2) adds Node.js/Express server and MongoDB. All phones share same DB via REST APIs.
Audience: Instructor, TAs, Developer.

1.2 Scope (Updated for A2) - IN SCOPE V2.0
Backend is now IN scope. Frontend from A1 stays.

In Scope:
- Real Auth with bcrypt + JWT (POST /api/auth/register, login)
- Menu from server GET /api/menu, not mock file
- Search, Sort, Debounce same UI but data from server
- Cart with promo WELCOME10, FEAST20 - server validates
- Order Summary: subtotal + 5% service + 15% tax - discount, total recalculated server-side
- Reservation: POST /api/reservations with server clash check 409
- Order Placement POST /api/orders, Tracking via polling GET /api/orders/my every 10s
- Manager Dashboard: GET /api/orders, PATCH status, reservations, menu POST/PUT/DELETE protected by protect + managerOnly
- Backend: server/config, models, routes, controllers, middleware, server.js cors(), express.json(), /api/health, 404, 500
- DB: MongoDB restaurant_app with 5 collections seeded
- Connected App: src/api/client.js BASE_URL = laptop IP not localhost, emulator 10.0.2.2:5000

Problems Noted in A1 Testing and Fixes (Required by A2 Brief):
P1: Manager price change not visible to others -> Fix: MenuItems in MongoDB, all GET /api/menu
P2: Order on phone A not on B -> Fix: Orders collection with userId, GET /api/orders/my
P3: Reinstall deletes all data -> Fix: MongoDB persistence
P4: Double booking same table same time -> Fix: Server returns 409 Conflict
P5: Client can hack total -> Fix: Server recalculates total
P6: No real security -> Fix: JWT + bcrypt + protect middleware

Out of Scope Still: Real payment, push notifications, image upload, multi-restaurant, Redux. Backend now IN scope.

1.3 Definitions (14 new terms for A2)
API - endpoints like /api/menu
REST - GET POST PUT PATCH DELETE
Endpoint - POST /api/auth/login
JSON - req/res format
MongoDB - NoSQL DB restaurant_app
Collection - Users, MenuItems etc
Mongoose - ODM for schemas
Middleware - cors(), auth protect, error handler
JWT - Bearer token Authorization: Bearer <token>
HTTP Status - 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Error
CORS - allows app to call server
bcrypt - hashes passwords
BASE_URL - http://192.168.1.15:5000 (laptop IP) - phone localhost != laptop
Seed - npm run seed populates DB

5. DATABASE DESIGN (MongoDB) - Replaces A1 Section 5 Mock Data
DB Name: restaurant_app
Connection: config/db.js mongoose.connect(MONGO_URI), log MongoDB Connected, exit on fail

Collection: Users
Fields: fullName String required, email String required unique lowercase trim, password String requi
red hashed select:false, role String enum [customer, manager] default customer
Relationships: One User has many Orders, Reservations. email unique index. Pre-save bcrypt hash. Met
hod matchPassword()

Collection: MenuItems
Fields: name String required, description String, price Number required min 0, category enum [Starte
rs, Mains, Desserts, Drinks] required, image String, isSpecial Boolean default false, isAvailable Bo
olean default true, createdBy ObjectId ref Users
Relationships: Referenced by Order.items.menuItem. Public read, manager write. Index on category.

Collection: Tables
Fields: number Number required unique, seats Number 1-12 required, isActive Boolean default true, lo
cation String
Relationships: One Table has many Reservations. number unique. Used for availability check.

Collection: Reservations
Fields: user ObjectId ref Users required, table ObjectId ref Tables required, date Date required, ti
meSlot String required e.g. 19:00-20:00, partySize 1-12 required, phone String required, status enum
[pending, accepted, declined, cancelled] default pending
Relationships: Refs Users, Tables. Compound index table+date+timeSlot unique -> 409 if double booked
. Check seats >= partySize.

Collection: Orders
Fields: user ObjectId ref Users required, items: [{menuItem ObjectId ref MenuItems required, name, p
rice, qty min1, note}], total Number required calculated server-side, type enum [dine-in, takeaway]
required, table ObjectId ref Tables, pickupTime String, status enum [Pending, Preparing, Ready, Serv
ed, Cancelled] default Pending, createdAt Date
Relationships: Refs Users, MenuItems, Tables. total NOT trusted from client. Status transition only
forward: Pending->Preparing->Ready->Served, validated in controller else 400.

6. Non-Functional V2 Updates
Security: bcrypt, JWT 7d expiry, protect middleware, .env not committed
Performance: Server <500ms for GET /api/menu, lean() queries, FlatList, useMemo
Reliability: db.js exit on fail, error handler JSON 500, interval cleared
Network: BASE_URL laptop IP, handle Network request failed, retry

8. API ENDPOINTS (New Section for A2 - 17 Endpoints, Requirement 15+)
Base URL: http://<laptop-ip>:5000/api - Postman localhost:5000/api, Phone laptop IP, Emulator 10.0.2.2:5000

Method | Endpoint | Access | Request Body | Response

POST /api/auth/register | Public | fullName, email, password, role | 201 {token, user} or 400 email exists
POST /api/auth/login | Public | email, password | 200 {token, user} or 401 invalid
GET /api/menu | Public | None ?category | 200 List MenuItems
GET /api/menu/:id | Public | None | 200 Single or 404
POST /api/menu | Private ManagerOnly | name, description, price, category, isSpecial, isAvailable | 201 Created or 401/403
PUT /api/menu/:id | Private ManagerOnly | fields to update | 200 Updated
DELETE /api/menu/:id | Private ManagerOnly | None | 200 Deleted
POST /api/orders | Private Customer | items [{menuItemId, qty, note}], type, tableId/pickupTime | 201 Order, total server-calculated
GET /api/orders/my | Private Customer | None token | 200 My orders
GET /api/orders | Private ManagerOnly | None | 200 All orders populated
PATCH /api/orders/:id/status | Private ManagerOnly | status | 200 Updated or 400 invalid transition
POST /api/reservations | Private Customer | tableId, date, timeSlot, partySize, phone | 201 or 409 if booked
GET /api/reservations/my | Private Customer | None | 200 My reservations
GET /api/reservations | Private ManagerOnly | None | 200 All reservations
PATCH /api/reservations/:id | Private ManagerOnly | status accepted/declined | 200 Updated
GET /api/health | Public | None | 200 {status: ok, time: ISO}
GET /api/tables | Public | None | 200 List tables

For any URL that does not exist, return 404 with message Route not found. Error handler sends JSON 500.

7. UML DIAGRAMS UPDATED FOR A2

Figure 3: Sequence Diagram - Customer Places an Order (V2) - 3 Lifelines: App, Express Server, MongoDB

App -> Server -> MongoDB:
1. App: POST /api/orders with Authorization Bearer token, body {items, type, tableId}
2. Server auth middleware verifies JWT, extracts userId, if fail 401
3. Server orderController validates, MenuItem.find({_id in items}) from MongoDB
4. MongoDB -> Server: returns prices
5. Server recalculates total = sum(price*qty) + 5% service + 15% tax - discount server-side
6. Server -> MongoDB: Order.create({user, items, total, status: Pending})
7. MongoDB -> Server: saved order
8. Server -> App: 201 Created {order}
9. App polling: useEffect setInterval 10s GET /api/orders/my with token -> Server -> MongoDB Order.find({user}) -> App updates UI when manager changes status to Ready. Interval cleared in cleanup.

Figure 4: State Machine: Order Lifecycle (same states, now enforced server-side)
Pending -> Preparing -> Ready -> Served, Cancelled from Pending only. Transition via PATCH /api/orders/:id/status, server validates only forward, else 400.

Figure 6: Deployment Diagram (New for A2)

Phone (Expo Go / React Native App) contains src/screens, src/api/client.js BASE_URL = http://<laptop-ip>:5000
  |
  | HTTP/WiFi same network
  v
Laptop (Node.js Express Server port 5000, IP 192.168.1.x) contains server/server.js, routes, controllers, models, .env not committed, .env.example committed
  |
  | Mongoose mongoose.connect(MONGO_URI)
  v
MongoDB Atlas or local Compass DB restaurant_app

Note: Phone localhost != laptop, must use IP. ipconfig (Windows) or ifconfig (Mac). Emulator uses 10.0.2.2:5000. If Network request failed, check WiFi and firewall port 5000.

9. Implementation Notes

server folder: npm init, install express cors dotenv, nodemon dev, folders config models routes cont rollers middleware

.env: PORT=5000, MONGO_URI, JWT_SECRET, in .gitignore, .env.example committed

config/db.js: mongoose.connect, log MongoDB Connected, catch error process.exit(1)

Seeding: customer@test.com / 12345678 and manager@test.com / 12345678, 6 tables, 15+ items

Client: src/api/client.js helper adds Content-Type and Authorization Bearer token

Deliverables: server folder, .env.example, Postman collection, screenshots health, SRS_v2.pdf in A2 folder